

Xiomara Strayhorn
Software Engineering — Starter Project #3
Review Partner: Nydeef Taylor

Code Review Summary:

Overview

The point of this assignment was to improve my Boggle solver through peer review, static analysis, and regression testing. My solver finds dictionary words by searching through adjacent board tiles, including diagonals, without reusing a tile twice in one word. It also supports the multi-letter tiles: Qu, St, and Ie. In my implementation, a trie is used to organize dictionary prefixes, allowing the search to stop early when a path is unable to produce a valid word. For the review process, my work was on a separate GitHub branch, and I opened a pull request. Nydeef reviewed my implementation, and I reviewed his.

Feedback I Received

Nydeef mainly focused his feedback on making the code easier to read and maintain. His first suggestion was to use Python's snake_case naming convention, such as `set_grid()` instead of `setGrid()`. Second, he recommended I change the dictionary variable `words` to a more descriptive name, such as `word_map`, to distinguish it from the trie. Finally, he suggested explaining how the visited set works during recursion. Adding and removing positions from this set is important because a tile cannot repeat within one path, but it must remain available for other paths. Overall, his review recognized the organization, input validation, and efficiency of the trie-based approach.

Feedback I Gave

In my review of Nydeef's code, I focused on possible correctness issues and useful test cases. I noticed his solver calculated results in `__init__`, but changing the grid or dictionary afterward did not recalculate them. Therefore, I suggested updating the results so `getSolution()` would reflect the current inputs. I also found duplicate words were removed before conversion to uppercase. Entries such as "cat" and "CAT" could later become duplicate results. My last suggestion was to check that single-character tiles were letters, since numbers and punctuation could pass the existing length check. I submitted three inline comments and an overall summary that also acknowledged his clear comments and handling of multi-letter tiles.

Improvements Implemented

Since the revision, my solver addresses all three suggestions from Nydeef. The setter implementations currently use `set_grid()` and `set_dictionary()`. However, I kept `setGrid()` and `setDictionary()` as wrappers that call the new methods. This improves the names while also

preserving compatibility with existing callers and the assignment's interface. I also renamed words to `word_map`, making its role clearer, thus connecting normalized words to their original dictionary spelling. Comments beside `visited.add()` and `visited.remove()` now explain that a tile is marked for the current path and released during backtracking. Before these changes, I preserved the original solver as `boggle_solver.py.orig`. The updated version was committed to my review branch, and another review was requested.

Static Analysis Results

Static analysis proved to be beneficial in distinguishing stylistic concerns from functional performance. When running `pycodestyle`, it revealed a solitary line in the solver's sample dictionary that exceeded the prescribed length constraint. Reformatting the list onto a separate line eliminated the linting warning while preserving the same logic. The initial starter test file in Codio contained a combination of tabs and spaces, resulting in a `TabError` that stopped the execution of tests. Furthermore, that test file depended on placeholder validations that did not adequately assess the solution. The revised unit tests rectify the indentation problems and substitute placeholder assertions with actual test cases. As a result, both the Analyze Solver and Analyze Unit Tests validations in Codio were successful.

Regression Testing

The regression testing checked for readability improvements while preserving the solver's behavior. Before the GitHub update, there were ten local tests passed, including a test comparing the revised solver with the original across 200 reproducible generated cases. Using Codio, the official correctness assessment passed all 26 tests. The expanded `tests.py` file also passed all 17 tests. These tests cover diagonal movement, backtracking, preventing tile reuse, duplicate words, invalid inputs, and whole-tile handling of Qu, St, and Ie. Also, they checked that changing the grid or dictionary updates the results so that larger boards work correctly. Working together, these checks provide evidence that the refactoring preserved the tested functionality.

Reflection

This review process has shown me that even working code can benefit from clearer names and explanations. A little comment about backtracking can make a recursive algorithm easier to follow for your reviewer. Another implementation showed how the operations can interact. For example, if you first remove duplicates and then normalize the case of letters, you can get a different result than if you do those steps in reverse order. It also taught me that improvement naming should respect existing interfaces and that the original setter names should be kept as is. Going forward, I'd check for input changes and edge cases earlier, run style checks frequently, and ensure review comments are specific enough that the suggested improvement will be easy to understand and verify.